



THE CITY OF VAEL

UTILITIES

04 – APIs and Integrations Review

What is hooked up, what is labeled on the panel, and what does not exist yet.

PREPARED FOR

Tony Grant, Owner
TSG Construction LLC

DOCUMENT TYPE

Milestone 1 deliverable
PRD acceptance pack

PREPARED BY

Appifest – Technical Audit
Continuing the existing prototype

DATE / METHOD

25 August 2026
Read-only inspection of the live tree

1. Utilities in the PRD sense

Utilities are APIs, edge functions, and third-party connections. For a City that is meant to match people in real time across devices, this is the electrical and plumbing. Today the building has labeled breaker panels and almost no live feed.

2. Inventory

UTILITY	STATUS	EVIDENCE
Live database / Auth project	Not connected to the running app	<code>supabase = null</code> without env vars. No <code>.env</code> in this tree.
Supabase JS SDK	Listed dependency; inert at runtime	<code>package.json</code> ; <code>src/integrations/supabase/client.ts</code>
Edge functions	None	No <code>supabase/functions/</code> directory
File storage bucket	Disabled / unused	<code>supabase/config.toml</code> storage enabled = false. Media <code>storage_reference</code> stays null.
SMS	Not connected	<code>notificationChannels.ts</code> : only <code>in_app</code> is true. Label: NOT YET CONNECTED. Local consent only.
Email	Not connected	Same channel map. Mock “resend verification” does not send mail.
Web push / native push	Not connected	Same channel map. No Firebase/APNs.
Payments / Stripe	Not present	Zero Stripe references in <code>src/</code> . Extended VAEL checkout is a local demo. <code>visibility_plans.is_purchasable</code> defaults false.
External content sources	Placeholders	<code>sourceRegistry.ts</code> uses <code>example.com</code> URLs. No live fetch of third-party sites.
Concierge	Local rules engine	<code>concierge.ts</code> – keyword routing, explicitly not a live AI service, no network call.
CI (GitHub Actions)	Not in this tree	No <code>.github/</code> . SQL test README mentions a workflow file that is not present here. 0017 cites an external <code>infra/vael-db-validator</code> branch/folder that is not in this repository.
Deploy (Netlify / Vercel / Docker)	None	No <code>netlify.toml</code> , <code>vercel.json</code> , or Dockerfile.

UTILITY	STATUS	EVIDENCE
Git remote	None in this copy	This workspace has no <code>.git</code> directory. Architecture notes say historically there was no configured remote.

3. Notification plumbing

Five channels are named: in-app, SMS, web push, email, native push. Only in-app is marked connected. Screens that offer channel toggles are required to read `CHANNEL_CONNECTED` / `channelStatusLabel()` rather than pretending a vendor exists.

On the Foundation drawing, `notification_events` can hold in-app records. `notification_deliveries` is the meter for actual sends – and nothing in SQL or TypeScript writes to it. That is honest incompleteness, not a hidden send path.

The notification service is implemented against those tables and is **not wired to any screen**. The bell, notification pages, and lifecycle notices still read/write `mockNotifications.ts` (and related preference/consent mocks) directly.

4. Media plumbing

What users attach in Handshake chat today goes through `mockAttachments.ts` (metadata in localStorage, preview as a session object URL). A parallel media service writes a different local key and is unused by those screens. The Supabase media provider can create a `pending` metadata row and cannot mark a file stored: that function is service-role-only, and no callback/edge function exists to call it. End to end, there is no Utility that holds file bytes in the cloud.

Profile documents in local mode store file bytes as a data URL on the record – a documented exception. That will not survive a Foundation cutover without a storage Utility.

5. Payments (explicitly out of scope)

The PRD puts payment processing and in-platform commissions out of scope unless a Change Order is approved. Finalizing the subscription model is also out of scope. What exists is structural demo only:

- Extended VAEL plans, checkout, and success pages (protected files).
- A `DEMO PRICING – NOT YET AVAILABLE FOR PURCHASE` label on paid tiers.
- SQL `price_demo_cents` / `is_purchasable = false`.

Milestone 3's "subscription gating, structural only" can reuse these hooks. It must not quietly become Stripe.

6. Dead Utility: exchange_listings

`src/data/listings.ts` calls `supabase.from("exchange_listings")`. That table is not in migrations 0001–0020. The live listing Room is `mockListings.ts` / drawn table `vaels`. `useListings.ts` imports the dead module; no page imports the hook. This is leftover original-plan plumbing and the source of three of the seven known TypeScript errors.

7. Local verification Utility

The repo does include developer-side inspectors: `npm run verify` (`dev-agent/verify.mjs`) and `npm run vael:db-verify` (`scripts/vael-db-verify.mjs`). SQL tests under `supabase/tests/` are design-review assertion blocks (not evidence they have been run against a live DB in this copy). They are not a substitute for CI or a deploy checklist (Milestone 4).

8. Third-party risk summary

DO NOT CONNECT FROM THIS MILESTONE

The project rule is explicit: never push, publish, or connect SMS, email, push, storage, database, or hosting without separate Owner approval. This audit did not connect anything. Leftover CLI metadata naming `vael-media-tech-dev` should be treated as a credential-hygiene / Owner-confirmation item (Punch List), not as an invitation to resume that link.

9. Utilities verdict

In-app notifications, local file previews, and a rules-based concierge are the only “utilities” a user actually feels – and they are all in-process, not networked. Everything the PRD will eventually need for a real City (shared Foundation, storage, outbound messaging, deploy, CI) is either a drawing, a labeled demo, or absent. That is appropriate for a prototype. It is also why Milestone 4’s deployment checklist does not exist yet, and why Milestone 2 should not start with “turn on Supabase.”